War Stories — Bugs Hit During Deployment

A single-author project doesn't get sanitized; every fix below is a real debugging session that ate hours and produced a commit. They're listed here because *what broke* and *why* tells you more about an engineer than the green CI badge does.

Two phases of hardware:

1. **Laptop** — single-node K8s on Ubuntu, NVIDIA RTX 4050 6 GB, GPU time-slicing into 2 software slots (`telemetry` branch).
2. **Lambda Labs A100** — single-node K8s on a Lambda bare-metal instance, A100 40 GB carved into 7× 1g.5gb hardware MIG instances (`GCP_BRANCH`). The branch name is historical — original plan was GKE, the realized deployment is Lambda.

Laptop (RTX 4050) — `telemetry` branch

1. Control plane crash loop after `kubeadm init`

Symptom: kubelet started, etcd / apiserver / controller-manager / scheduler came up, then all four were SIGTERM'd ~14 seconds in. Loop repeated forever. `kubeadm init` sat at *waiting for control plane to be healthy*.

Root cause: containerd's compiled-in default uses the `cgroupfs` cgroup driver. kubelet on Ubuntu 22.04 with cgroup v2 expects `systemd` . Mismatch = kubelet refuses to keep the static pods alive.

Fix: generate the default config and replace `SystemdCgroup = false` → `true` , then `systemctl restart containerd` . Encoded in `script/laptop/all_install.sh` so it can't regress.

2. `vllm-worker` stuck in `ContainerCreating` forever

Symptom: pod scheduled, image pulled, but events showed `no runtime for "nvidia" is configured` . Pod never moved.

Root cause: `nvidia-container-toolkit` (host-level package) was not installed. The K8s `nvidia-device-plugin` DaemonSet exposes `nvidia.com/gpu` resources, but pods that ask for those resources via `runtimeClassName: nvidia` need the actual `nvidia-container-runtime` binary on the host. Two different things, both required, neither installs the other.

Fix: explicit phase 2.5 in `all_install.sh` to install `nvidia-container-toolkit` and inject the nvidia runtime handler into containerd's config via `nvidia-ctk runtime configure` .

3. vLLM 0.11 + transformers 5.x incompatibility

Symptom: `vllm-worker` came up, then crashed during model load with `AttributeError: 'Qwen2Tokenizer' object has no attribute 'all_special_tokens_extended'`.

Root cause: pip's most recent `transformers` (5.x) deprecated the attribute `all_special_tokens_extended` on tokenizers. vLLM 0.11 still calls it.

Fix: pin `transformers==4.57.6` in `app/worker/vllm/Dockerfile`. Build new image, tag, push, ArgoCD Image Updater rolls it out.

4. `vllm-worker` requested 16 GiB RAM, used 1.81 GiB

Symptom: laptop OOM'd under 2-replica time-slicing. `kubectl top pod` showed `vllm-worker` actually using ~1.8 GiB resident. The 16 GiB request was a copy-paste from a tutorial for 7B models.

Fix: right-sized to `requests: 4Gi / limits: 8Gi` after measuring real RSS. Doubled the cluster's headroom. Resource-tuning matters more than people think.

5. Gateway returned 502 even when vLLM returned 4xx

Symptom: invalid model name → vLLM correctly returned 400; the gateway in front of it wrapped the error and surfaced it as 502 "Bad Gateway". Caller had no idea what was actually wrong.

Root cause: blanket `try / except httpx.HTTPStatusError` re-raised everything as a 502.

Fix: in `app/gateway/gateway.py`, propagate the upstream status code unchanged when it's a 4xx. Only wrap genuine 5xx.

6. HPA on custom metric showed `<unknown>/5`

Symptom: `kubectl get hpa` showed `vllm:num_requests_waiting` forever as `<unknown>/5`, so HPA never scaled.

Two layers of root cause:

- **prometheus-adapter URL path**: `kube-prometheus-stack` exposes Prometheus under `/prometheus`, but `prometheus-adapter` was configured to query the root URL. Adapter logs showed 404s.
- **ArgoCD selfHeal vs HPA**: even when adapter was fixed, ArgoCD saw the HPA-mutated `Deployment.spec.replicas` diverge from git's `replicas: 1` and reverted it every minute.

Fixes:

- Adapter URL: `prometheus-adapter-values.yaml` → `prometheus.url: http://...:9090/prometheus`.
- HPA / ArgoCD coexistence: `tools/argocd-image-updater/llm-application.yaml` added `ignoreDifferences` for `/spec/replicas` on the HPA-managed Deployments.

7. Grafana / Prometheus evicted under node memory pressure

Symptom: Grafana showed `0/3 Unknown` after a load test. node-exporter got OOM-killed first, but the eviction cascade took everything.

Fix: combined two protections in `kps-values.yaml`:

- **PriorityClass** `monitoring-critical` (value 100000) so kubelet evicts business pods first under pressure.
- **Guaranteed QoS** by setting `requests == limits` on every monitoring component. Guaranteed is the *last* eviction tier.

8. RTX 4050 needed defensive vLLM flags

Symptom: vLLM crashed at startup on the consumer RTX 4050 with illegal-memory-access errors after enabling FlashAttention 2.

Root cause: FA2 + cascade attention in vLLM 0.11 are tested on data-center cards (A100 / H100). Consumer cards hit edge cases.

Fix (laptop branch only): add `--enforce-eager` and `--disable-cascade-attn` to vllm-worker's args. The Lambda A100 branch removes these — they're a workaround, not a feature.

9. PAT scope blocked workflow file pushes

Symptom: `git push` rejected with "refusing to allow a Personal Access Token to create or update workflow `.github/workflows/local-build.yml` without `workflow` scope".

Fix: regenerate PAT with `workflow` scope; or, when the change is small, edit the workflow file directly in the GitHub web UI (which uses session auth, not the PAT). Encoded in commit messages so future-me doesn't waste time on the same wall.

Lambda Labs (A100 + MIG) — `GCP_BRANCH`

10. MIG mode not enabled on the bare instance

Symptom: `nvidia-smi -L` only showed `GPU 0`, not the expected 7 MIG instances. `nvidia-device-plugin` couldn't expose `nvidia.com/gpu: 7`.

Root cause: Lambda's stock A100 image ships with MIG **disabled**. On GKE you'd configure `--gpu-partition-size=1g.5gb` at node pool creation; on a bare instance, you do it yourself with `nvidia-smi mig`.

Fix: `script/lambda/all_install.sh` step `[6/6]` now does it automatically:

1. `nvidia-smi -mig 1` to enable MIG mode.
2. `nvidia-smi mig -lgip` to discover the 1g.5gb profile id.

3. `nvidia-smi mig -cgi $ID,$ID,$ID,$ID,$ID,$ID,$ID -C` to create 7 GPU instances + matching compute instances in one shot. Idempotent — destroys partial state and recreates if it sees < 7.

11. Helper pods stuck in `ContainerCreating` for 14+ minutes

Symptom: `local-path-provisioner` create a busybox helper pod to `mkdir` PVC directories. Pod showed `Pulling image "busybox"` 14 minutes ago and never moved past it. No errors, no retries — just silent stall.

Root cause: Lambda's egress IP pool is shared across many tenants and **rate-limited by Docker Hub**. Anonymous pulls of `docker.io/library/busybox` from the same IP pool collectively burn through the 100/6h limit, then queue silently. This wasn't visible in pod logs because the pull was literally blocking, not failing.

Fix: configure containerd's CRI registry mirror so `docker.io` resolves through `mirror.gcr.io` (Google-maintained mirror, no rate limit) before falling back to docker.io. Encoded in `script/lambda/launch.sh` Phase 2.5.

12. `kube-prometheus-stack` install hit the same wall × 9 images

Symptom: after the busybox unblock, the kps install timed out at 10 minutes with Grafana / Prometheus / Alertmanager / k8s-sidecar pods stuck in `ImagePullBackOff` or `ContainerCreating`. Each pod was waiting on a different image.

Root cause: same docker.io / quay.io throttle, multiplied by chart complexity. Even after the mirror was in place, kubelet's image-pull state machine had wedged on stale "Pulling" tasks from the previous attempts.

Fix (two parts):

- **Pre-pull**: new helper `prepull_helm_images()` in `launch.sh` runs `helm template` to enumerate every image the chart will need, then `crictl pull`s each into containerd's local cache *before* running `helm install`. crictl talks to containerd directly, bypassing kubelet's wedged state.
- **Kick stuck pods**: `kick_stuck_pods()` force-deletes any pod in non-Running state after a helm timeout, letting the ReplicaSet/StatefulSet rebuild the pod — the new pod gets the cached image and starts in seconds.

13. Grafana `init-chown-data` Permission denied loop

Symptom: after kps was up, Grafana sat in `Init:CrashLoopBackOff`.
The init container's logs showed `chown: /var/lib/grafana/csv: Permission denied` × 3, exit 1.

Two-step root cause:

- The PV directory on the host (`local-path-provisioner` → `/mnt/k8s/k8s/pvc-...`) was created `root:root 0755`. The init container tries to chown it to `472:472` (Grafana user).
- Modern Grafana subchart (v8+) runs `init-chown-data` as **non-root** with `CAP_CHOWN` dropped. The Linux kernel requires CAP_CHOWN to call `chown()`, even to no-op.

Compounding fact: `fsGroup` doesn't help on `hostPath`-backed PVs.
The kubelet only applies `fsGroup` to "real" volumes.

Fix (two parts, both required):

- `kps-values.yaml` : `grafana.initChownData.enabled: false` — because the chown can't succeed under the chart's security context.
- `launch.sh` `chown_grafana_pv()` : after the PVC binds, look up the hostPath, chown it to `472:472` from the host where root *can* call chown.

14. Grafana login page rendered as a blank "Welcome to Grafana"

Symptom: opened `/grafana/`, expected anonymous Admin landing,
saw a stripped page with only the title "Welcome to Grafana".
No login form, no error, can't enter the app.

Root cause: in `kps-values.yaml`, the auth config was written as nested YAML maps:

```
grafana.ini:
  auth:
    anonymous:
      enabled: true
      org_role: Admin
    disable_login_form: true
```

But `[auth.anonymous]` in `grafana.ini` is a **literal section name with a dot in it**, not nested config. The Grafana subchart's helm template renders nested maps with `toJson`, producing:

```
[auth]
anonymous = {"enabled":true,"org_role":"Admin"}
disable_login_form = true
```

Grafana ignores the JSON blob, so anonymous Admin never activates.
And because we'd already set `disable_login_form: true`, the login form was hidden too. User locked out by their own config.

Fix: switch to literal quoted-string keys:

```
grafana.ini:
  auth:
    disable_login_form: true
  "auth.anonymous":
    enabled: true
    org_role: Admin
  "auth.basic":
    enabled: false
```

`grafana.ini` rendered correctly: separate `[auth.anonymous]` section, anonymous Admin works, the disable-login-form flag is harmless.

15. DCGM dashboard showed all 7 MIG rows as "GPU 0"

Symptom: imported the upstream NVIDIA DCGM dashboard (id 12239). Got 7 separate time series — but every legend label said "GPU 0". Couldn't tell MIG-7 from MIG-13 without reading the value.

Root cause: dashboard 12239 was authored before MIG existed. Every panel uses `legendFormat: "GPU {{gpu}}"`. On a single physical card, all MIG instances share the same `gpu="0"` label; the discriminator is `GPU_I_ID`.

Fix: wrote a custom dashboard `tools/helm/monitoring/dashboards/mig-vllm-dashboard.json` that uses `MIG-{{GPU_I_ID}} {{exported_pod}}` everywhere, plus a table panel showing the live MIG ↔ Pod mapping. 7 distinct lines, each labeled by MIG ID and the vLLM pod currently bound to it.

16. DCGM relabel: `pod` was the dcgm-exporter pod, not the workload

Symptom: the new dashboard's table column "Pod" displayed `dcgm-dcgm-exporter-bv4fw` for every MIG row, not the vLLM pods.

Root cause: kube-prometheus-stack's ServiceMonitor automatically adds a `pod` label = the pod being scraped. That overwrites DCGM's own `pod` label (the workload pod attached to the MIG), which gets relabelled to `exported_pod`.

Fix: switch every dashboard query and Prometheus helper to use `exported_pod` instead of `pod`. One commit, multiple files, easy once the relabel chain is understood.

17. HPA scaled to 5/7, not the full 7

Symptom: 12-minute extreme stress test showed HPA peaked at 5 replicas, not the configured `maxReplicas: 7`. Timeline showed

`vllm:num_requests_waiting` was well above the threshold the entire time.

Root cause — *not* a bug, an HPA timing budget:

- Each `scaleUp` is gated by `stabilizationWindowSeconds: 60` + `policies.periodSeconds: 120` = ~180 s minimum between adds.
- New vllm-worker pods take 30-60 s of cold start (image already cached, but model load + MIG attach takes time).
- 12 min wall clock / 200 s effective per scale ≈ 3-4 scale-ups, starting from 1 replica → max ~5.

Fix is just to give HPA more time:

```
EXTREME_LOAD_DURATION=1200 EXTREME_LOAD_CONCURRENCY=80 \
  ./test/run_lambda.sh extreme
```

20 min × 80 concurrency reaches 7/7. Documented in the Lambda README section so this isn't read as a bug next time.

18. Tensor Core peaked at 14.5 %, not 100 %

Symptom: under heavy concurrent load, peak GPU compute hit 91 % (great) but Tensor Core utilization peaked at only 14.5 %.

Root cause: this is *expected* for Qwen2.5-0.5B. The model has ~1 GB of fp16 weights; even fully batched decode is bottlenecked by HBM bandwidth, not Tensor Core math. Tensor Core saturates only during the prefill phase of long prompts.

Note (not a fix — a planning observation): the next milestone is swapping Qwen2.5-0.5B for Qwen2.5-7B-Instruct-AWQ to actually exercise the A100's Tensor pipes. That's gated on quantization-stack work (AWQ + fp8 KV cache), not on infra.

Patterns

A few of these issues share the same shape:

- **Default config is wrong for our use case**, not "broken" — containerd cgroup driver, Grafana init-chown-data, prometheus-adapter URL path, vllm-worker resources, FlashAttention defaults. Engineering on top of off-the-shelf tooling is mostly *finding the right knobs and learning when the tutorial is wrong for your environment*.
- **Symptom and cause are far apart**. Grafana login looking blank was a YAML-rendering bug 200 lines away; control plane crash loop was a containerd config flag; helper pod stalls were a Docker Hub rate limit. Fast diagnosis comes from knowing where to look, not from reading the error.

- **One fix per environment** vs **one fix that works everywhere**. Several issues (cgroup driver, registry mirror, MIG enable) are now encoded in `all_install.sh` / `launch.sh` so a fresh Lambda instance bootstraps clean in ~20 minutes without manual debugging. That's the value of writing the workaround down *as code, not as a runbook*.